The tools and techniques of R programming Lesson-13 Searching and traditional Data Structures implemented in R

By
Prof.Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
Email-Id:hod.ca@technoindiaeducation.com

January 18, 2022

SEARCHING

The term searching refers to the task of finding out the location of some given value in a collection of data elements in the main memory of computer. The search is said to be successful if the given value does appear in the collection of data and unsuccessful,otherwise. There are two searching methods used in such a situation, namely,

1. **Linear search** and
2. **Binary search**

The linear search is the method of traversing the locations of the data sequentially to locate the given value.If it is found at any location, the location number is returned; otherwise, message is given that the data does not exist in the given set of values. As the locations are checked one after another until the value is found or the end of the list is reached, the search is popularly known as **sequential search**.

The complexity of the linear search algorithm can be measured by the number of comparisons required to find out the given value in the set of data values stored in memory.

SEARCHING–Contd.

If we assume that there are n values of data elements stored in the main memory, then we may consider two cases of the count,namely,

1. The **average case** and
2. The **worst case**

Obviously, the worst case occurs when one must search through entire list of stored values and the search becomes unsuccessful. In this the algorithm requires : **f(n)=n+1 comparisons**.

Thus, in the worst case, the running time is proportional to n.

The running time of the average case can be calculated by using the probabilistic concept of expectation. Let p_i be the probability that the given value appears in the list for some i and q be the probability that it does not appear in the list. Hence, by probability theory, we shall have,

$p_1 + p_2 + \cdots + p_n + q = 1$

As the algorithm assumes i comparisons to find out the given value in the list, the average number of comparisons will be given by

$f(n) = 1.p_1 + 2.p_2 + 3.p_3 + \cdots + n.p_n + (n+1).q$

SEARCHING–Contd.

If we assume that q is very small and the probability of occurrence of the value being searched is same each of the locations, then q≈0, and p=$\frac{1}{n}$. So, we shall have,

f(n)=1. $\frac{1}{n}.+2.\frac{1}{n}.+3.\frac{1}{n}.+\cdots+n.\frac{1}{n}.+(n+1).0=(1+2+3+\ldots+n).\frac{1}{n}=\frac{(n(n+1))}{2}.\frac{1}{n}$.

Thus,f(n)=((n+1))/2, which is approximately half of the number of elements.

So, the time complexity can be expressed in **big oho notation as O(n)**.

The following function shows the process of sequential search discussed above.

```
# Function for sequential search
ssearch=function(v,e){
l=length(v)
flag=0
for(i in 1:l){
if(v[i]==e){
loc=i
flag=1
break}}
if(flag==1) return(loc)
else return(flag)}
```

Binary search

This algorithm has been devised with a view to reduce the number of searches in an array of data values. However, there is a pre-requisite of the application of this search process in an array and it is that the array of values must be in a sorted form. **The algorithm is also known as half-interval search, logarithmic search or binary chop**.

Beginning with an interval covering the whole array, the process narrows down the interval to the half of the previous interval if the mid value of an interval does not match with the value being searched. As the search is performed in a sorted array, the decision to focus on the search in which part of the array can be taken easily, if the mid value of the current interval does not match with the given value.

For example, let us consider an array having values in ascending order. Let BEGIN denote the position of the first value and END denote the position of the last value. So, the middle position (MID) of the array can be had by taking the integer part of (BEGIN+ END)/2. Now, the value at this position is checked with the value being searched. If it matches, we can conclude that the value occurs at MID position. CONTD· · ·

Binary search–Contd.

If it does not, then we compare the value with the value at the MID position. If the value being searched is greater than the value at the MID position, then it must be found in the bottom half of the array if it exists at all; otherwise, it must occur in the upper half of the array. So, we can narrow down our search interval. We then set the BEGIN to MID+1, if it can be supposed to occur in the bottom half, otherwise, the END is set to MID +1. Again, the MID location of the current interval is obtained to check the match. This is continued until either the value is found or END becomes less than the BEGIN, when we conclude that the value is not available in the array resulting in an unsuccessful search.

The complexity of the binary search algorithm can be obtained easily. If f(n) is the number of comparisons required to find out the value, then we must have,

2f(n) >n i.e. f(n)=$log_2 n + 1$

So, the complexity will be $O(log_2 n)$.

The following function implements the search process in R.

Binary search–Contd.

```
#Function for Binary search binary_search=function(v,e){
l=length(v)
vec=vector(mode="numeric",length=l)
vec=sort(v)
print(vec)
low=1;high=l;
while(low<high){
mid=ceiling((low+high)/2)
if(vec[mid]==e)
return(mid)
 else if (vec[mid]>e)
high=mid-1
 else
low=mid+1} if(vec[low]==e)
return(low)
else return(0)}
```

Linked Lists

A linked list is a linear arrangement of data elements called nodes where each node comprises two parts, the first holds the desired information and the second part called the link field contains the address of the next node in the list. It is these address –holding pointers in each node maintain the linear order in linked lists. The pointer of the last node of a linked list contains a special value called a null value and the pointer is known as a null pointer. This data structure can be accessed only sequentially starting from the head node of the list.So,to access any node from a linked list, we are required to access all its preceding nodes.The following figure shows a schematic diagram of a linked list.

In the following programs in R, we have implemented a linked list and its associated functions for searching a node in the list, inserting a node in the linked list, deleting a node from the linked list and displaying contents of a linked list.

Linked Lists

```
# Linked List Implementation Version-1
cat("How many nodes?")
m=scan(n=1)
lst=vector("list",m)
for(i in 1:m){
lst[[i]]=readline(prompt="Enter the values for your node of the linked
list")}
print("The content of the Linked list is: ")
for(i in 1:m)
print(lst[[i]])
```

Linked Lists–Contd.

```
#Linked List Version 2
lst=vector("list",10)
i=1
opt="y"
while(opt=="y"){
lst[[i]]=readline(prompt="Enter the values for your node of the linked
list")
i=i+1
opt=readline(prompt="continue?(y/n :")
}
print("The content of the Linked list is: ")
for(j in 1:(i-1))
print(lst[[j]])
```

Linked Lists–Contd.

```
#Linked Lists
repeat{
#cat("14");
#cat(rep("\n", 50))
print("Welcome to the program of Linked List")
print("Read carefully before entering your choice")
print("Enter 1 to create a list,2 to display the list,3 to search,4 to
insert 5 to delete, and 6 to terminate")
n=as.integer(readline(prompt="Your Option?->"))
switch(n,create_list(),show_list(),find_node(),insert_node(),delete_node
}
# creation of linked list
create_list=function(){
lst=vector("list",10)
i=1
opt="y"
while(opt=="y"){
```

Linked Lists–Contd.

```
lst[[i]]=readline(prompt="Enter the values for your node of the linked
list")
i=i+1
opt=readline(prompt="continue?(y/n :")
}
}
show_list=function(){
# Displaying the contents of the list
print("The content of the Linked list is: ")
j=1
while((length(lst[[j]]))!=0){
print(lst[[j]])
j=j+1
}
mywait()
}
```

Linked Lists–Contd.

```
find_node=function(){
# Searching the nth node from the list to see its content
cat("Which Node to search?")
n=scan(n=1)
print(paste("The Node is: ",lst[[n]]))
mywait()
}
insert_node=function(){
# Insertion in a Linked List
print("Where do you want to insert a node?  Enter the location number: \n")
n=scan(n=1)
lst[[n]]=readline(prompt="Enter the content")
cat("Node added successfully at the location",n,"of the list  its content is: ",lst[[n]])
mywait()}
```

Linked Lists–Contd.

```
delete_node=function(){
Deletion from the list cat("Which node to delete?")
n=scan(n=1)
print("The content of the list after the deletion is as shown below:")
lst[-n]
j=1
while(length(lst[[j]])!=0){
if(j!=n)
print(lst[[j]])
j=j+1
}
#mywait()
#b = scan("stdin", character(), n=1)
}
```

A stack is a linear data structure that represents a container of a set of objects which are inserted and removed according to **Last-In-First-Out(LIFO)** principle.

STACKS implementation through R

This **Abstract Data Type(ADT)** is defined with a view to mimic the real world handling of stacks as seen in hotels having a stack of plates, libraries having a stack of books etc.Only two operations are allowed in a stack,namely, the insertion operation, technically called a push operation and the removal or deletion operation ,technically called a pop operation. Both the push and the pop operation are limited to the top end of a stack. A stack may be called a recursive data structure because it is either empty or comprises a pointer to the top of the stack.

Applications of Stacks

- **The simplest application of a stack is to reverse a word**. We can push a given word to a stack - letter by letter - and then pop letters from the stack to the word in its reversed form.
- Another application is an **"undo" mechanism** in text editors; this operation is accomplished by keeping all text changes in a stack that can be popped up from the stack when the users express their wish to **"undo"**.
- **Backtracking**-This is a process of accessing the most recently stored data elements from a large set of elements. Such an operation is required in many situations such as backtracking from the dead end of a maze to the last point of selection.

Applications of Stacks–Contd.

- space for parameters and local variables is created internally using a stack.
- compiler's syntax check for matching braces is implemented by using stack.
- supports for recursion

Here is an implementation of stack in R.

```
Stack = setRefClass("Stack",
fields = list(
vStack = "vector",
nTop = "numeric"
 ),
 methods = list(
initialize = function() {
 nTop <<- 1  },
 push = function(nItem) {
vStack <<- c(vStack, nItem)
 nTop <<- nTop + 1
 vStack[nTop-1]
 },
```

Implementation of stack in R contd.

```
 pop = function() {
if (nTop == 1) return(NULL)
 nItem = vStack[nTop-1]
nTop <<- nTop - 1
 vStack <<- vStack[1:nTop-1]
 nItem
},
 top = function() {
 vStack[nTop-1]}))
stk=Stack()
 repeat{
print("Enter 1 to push,2 pop and 3 to terminate")
opt=as.integer(readline(prompt="Enter your option"))
switch(opt,pushing(),popping(),break)}
 pushing=function(){
 vl=as.integer(readline(prompt="Enter the value to push"))
 stk$push(vl)}
popping=function(){
 cat("The popped value is: ",stk$pop())}
```

The stack( ) and unstack( ) functions in R

The **stack() function** creates a stack of vectors by concatenating multiple vectors of a data frame into a single vector along with a factor indicating where each observation originated.
The **unstack( ) function** just does the reverse operation. The following R Codes illustrates the use of the functions where we have defined two vectors containing the same number of elements and then defined a data frame before using the functions under study.

```
v1=c(1,2,3,4)
v2=c(5,6,7,8)
d=data.frame(v1,v2)
>d
```

	v1	v2
1	1	5
2	2	6
3	3	7
4	4	8

The stack( ) and unstack( ) functions in R–Contd.

>**stack(d)**

	values	ind
1	1	v1
2	2	v1
3	3	v1
4	4	v1
5	5	v2
6	6	v2
7	7	v2
8	8	v2

>**unstack(d)**

	res
5	1
6	2
7	3
8	4

The syntax of the functions have been shown below:

The **stack()** is available in the following three forms:

- **stack(x,. . .)** # S3 method for default
- **stack(x, drop=FALSE, . . .)**
- **stack(x,select,drop=FALSE. . .)** # S3 method for data frame

The **unstack()** is available in the following two forms:

The stack( ) and unstack( ) functions in R–Contd.

- **unstack(x,. . .)** # S3 method for default
- **unstack(x,form,. . .)** # S3 method for data frame

S3 methods are functions with a special naming scheme, **generic.class()**. For example, the Date method for the **mean()** generic is called **mean.Date()**, and the factor method for **print()**is called **print.factor()**
Where the arguments possess the meanings stated below:

- **x** is a list or data frame to be stacked or **unstacked**.
- **select** is an expression, indicating which variable(s) to select from a data frame.
- **form** is a two-sided formula whose left side evaluates to the vector to be unstacked and whose right side evaluates to the indicator of the
- **groups** to create. Defaults to formula(x) in the data frame method for **unstack**.
- **drop** is logical implies, whether to drop the unused levels from the **"ind"** column of the return value.
- $\cdots$ implies further arguments passed to or from other methods.

QUEUES

A **Queue** is a linear data structure that represents a set of locations containing data elements which are accessed on the principle of First In **First Out(FIFO) or First Come First Served (FCFS)**.This **Abstract Data Type(ADT)** mimics the real world queues as observed in front of ticket counters, Bank counters etc. Hence, only two basic operations are defined for a queue, namely, insertion into a queue that must occur at the end of the queue (at the beginning both both ends can be considered to be same) and deletion or removal from a queue that must occur at the front end of the queue.So, two pointers are required to be defined for a queue; one for pointing to the front end of the queue and the other for pointing to the end of the queue. The former pointer is called Front or Head and the latter one is called the Rear or Tail. Using Front and Rear, we ensure that the elements that enter first in the queue will be the first candidate to be served for removal as in real life queues.

The process of adding an element into a queue is called **Enqueue** and the process of removal of an element from a queue is called **Dequeue.**

Basic characteristics of Queues are stated below:

QUEUES–Contd.

- It is an ordered list of elements of similar data types.
- Elements inserted into a queue are always prioritized to be served in the same sequence of their entry.
- Once an element is inserted into a Queue, all the elements inserted before the element in the queue must be removed in order to remove the element.
- A function called **Peep()** may be applied upon a queue to see the first element from a queue without dequeuing it.

Applications of Queue

Queue, as the name suggests ,is applied whenever we need to manage any group of objects in an order in which the first one taking entry in it, also gets out first while the others wait for their turns, as in the following scenarios: Requests on shared resources, like printers, CPU task scheduling , disk packs etc. are always served in a queue unless there is a well defined priority queue that works on the basis of an assigned priority and hence can supersede some prior entry if the priority is so assigned.

QUEUES–Contd.

In real life situations, Phone calling systems make use of Queues to hold people calling them in waiting state in the order of a queue, until a service representative becomes free for the service.

Handling of interrupts in real-time systems are also done in the order of their arrivals and hence queues are created to maintain the sequence. In R a queue can be created by defining a specialized function as shown below. The function has been named queue.

```
# Function to create a Queue
queue = function(){
 e=new.env()
 q=list()
 assign("q",q,envir=e)
class(e)=c("queue","environment")
 e }
```

Now, we define the following functions for the operations on a queue:

```
 push.queue=function(e,v){
q=c(v,get("q",envir=e))
 assign("q",q,envir=e)
 v }
```

QUEUES–Contd.

```
 pop.queue=function(e){
 q=get("q",envir=e)
 v=q[[length(q)]]
if(length(q)==1){
 assign("q",list(),e)
 }else{
 assign("q",q[1:(length(q)-1)],e) }
 return(v) }
 print.queue=function(x,...){
 print(get("q",envir=x)) }
```

We may keep the above definitions in a file ,say, implement_Q.R

Now, each time before using a queue, we should run the following command:

Source("implement_Q.R")

Now, we can make use of the functions to make our own queues as illustrated below: **Q=queue()**

push.queue(Q,12)

push.queue(Q,25)

pop.queue(Q)

etc.